

BRAINHACK 2026

RoboVerse: Flight Challenge
Search and Map

AGENDA

Main goal: To provide teams ideas/algorithms to extend obstacle avoidance capabilities into goal-based and search.

1. Goal + Avoidance Approach

2. Exploration Strategy

3. Object Detection

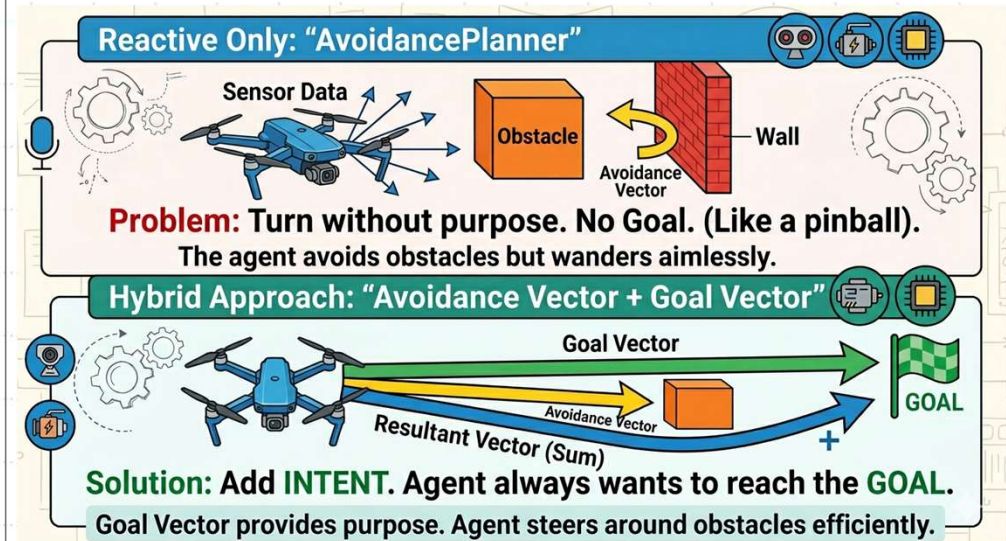
4. Occupancy Grid

1. Goal + Avoidance Approach

Goal vector

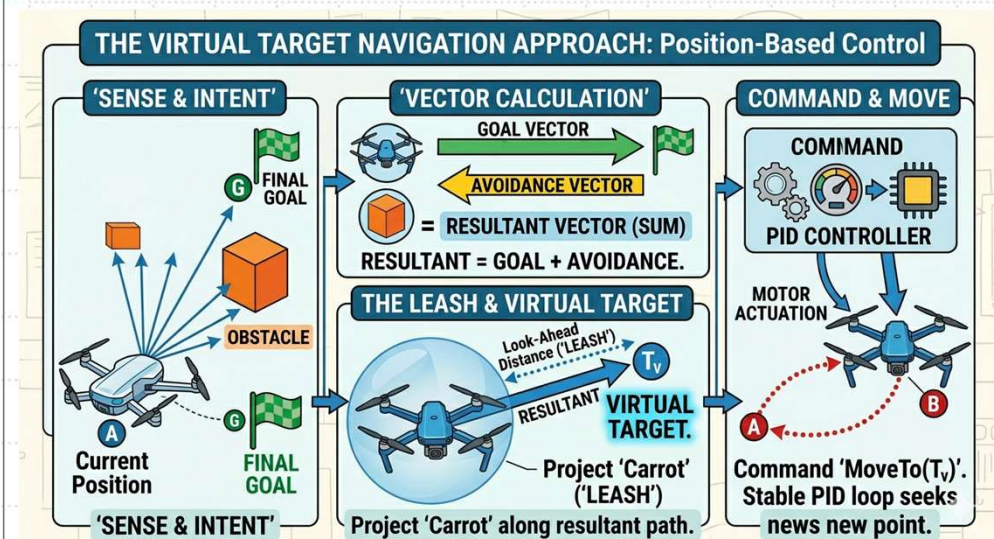
- AvoidancePlanner provides avoidance vector which telling where it is safe to go. This is very reactive.
- It is like pinball in pinball machine. When block, just turn. Quite aimlessly.
- The Goal Vector represents the "intent." It is a localized vector pointing from the agent's current position toward the target coordinates.
- Hybrid Approach: Avoidance Vector + Goal Vector
 - Goal Vector: Points directly at the door.
 - Avoidance Vector: Points away from the chair.
 - Resultant Vector: The agent "veers" slightly to the side to miss the chair but maintains its forward momentum toward the door.

Feature	Reactive (Avoidance Only)	Goal-Oriented (Avoidance + Goal)
Behavior	Turns away from obstacles randomly.	Steers around obstacles toward a target.
Success Rate	Low; may never reach the destination.	High; finds the most direct path.
Complexity	Simple "If/Then" logic.	Vector math (addition and weighting).
Analogy	A Roomba hitting a wall.	A person walking through a crowded hallway.



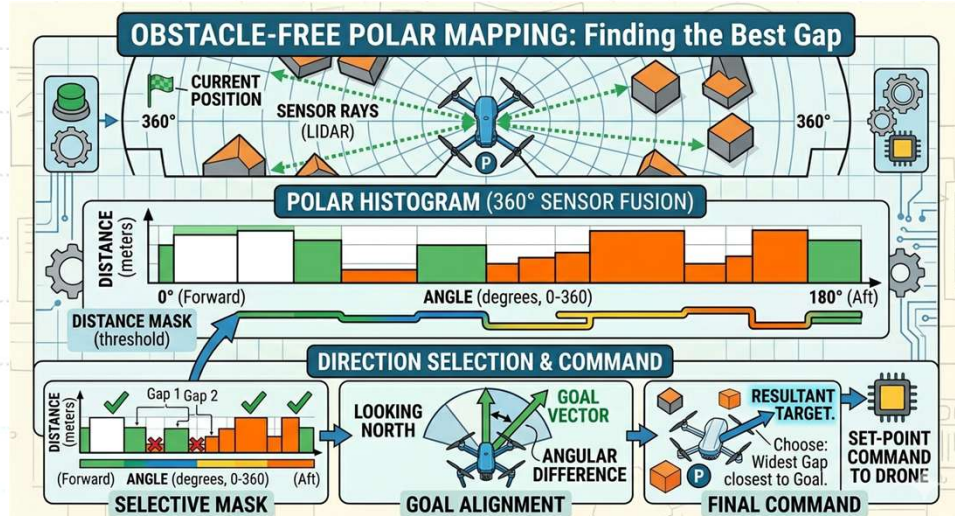
The "Virtual Target" Approach

- Let's say there is a list of positions that the drone should visit. Instead of sending goal positions to the drone, send Virtual Targets. The virtual target is the result of vectors:
 - Step 1: Calculate the Goal Vector (the direction to the target positions from current position).
 - Step 2: Calculate the Avoidance Vector (the direction away from obstacles or positions to avoid the obstacles).
 - Step 3: Sum them to get the Resultant positions.
 - Step 4: Project a point a short distance along that resultant direction. This point is your set_point for the next movement.
- The distance of the Look-Ahead (d) affects alot:
 - If d is too large: The drone may try to cut corners through obstacles because the target is on the other side.
 - If d is too small: The drone will move very slowly and jitter, as the position controller reaches the set-point too quickly.



Obstacle-Free Polar Mapping

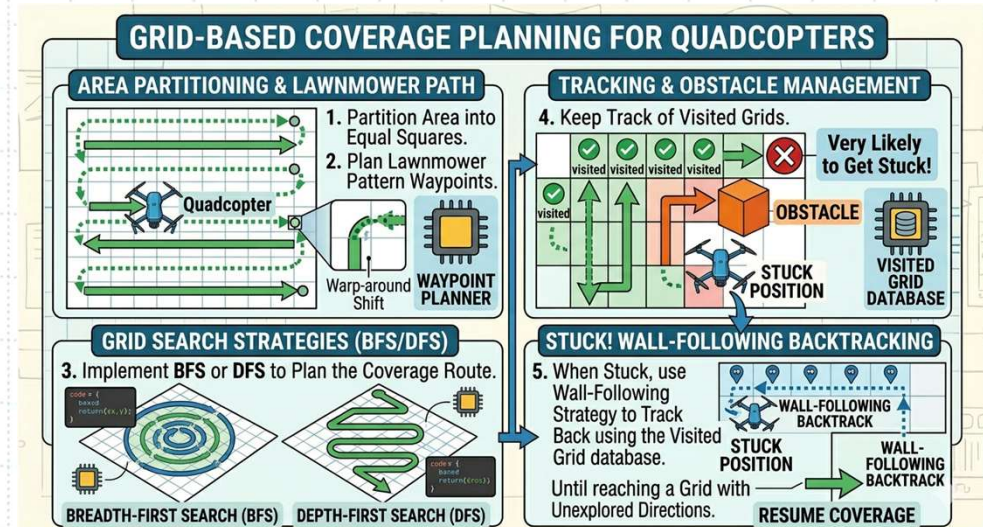
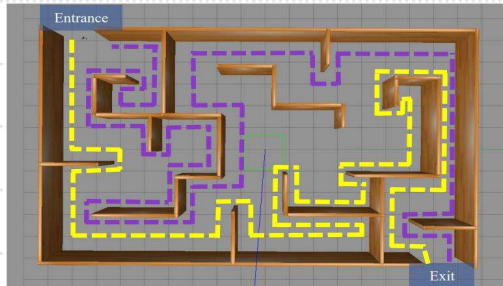
- The drone scans its surroundings as a 360° map of distances. Then:
 - Step A: The Depth Histogram. The drone divides its surroundings into "bins" (e.g., every 10°) as it yaw quarter by quarter.
 - Step B: The Mask. It marks bins as Blocked (if an obstacle is closer than a safety threshold) or Free.
 - Step C: The Selection. It looks at all the Free bins and chooses the one that has the smallest angular difference from the Goal Vector.



Exploration Strategy

Strategy to generate the Goals vectors (waypoints)

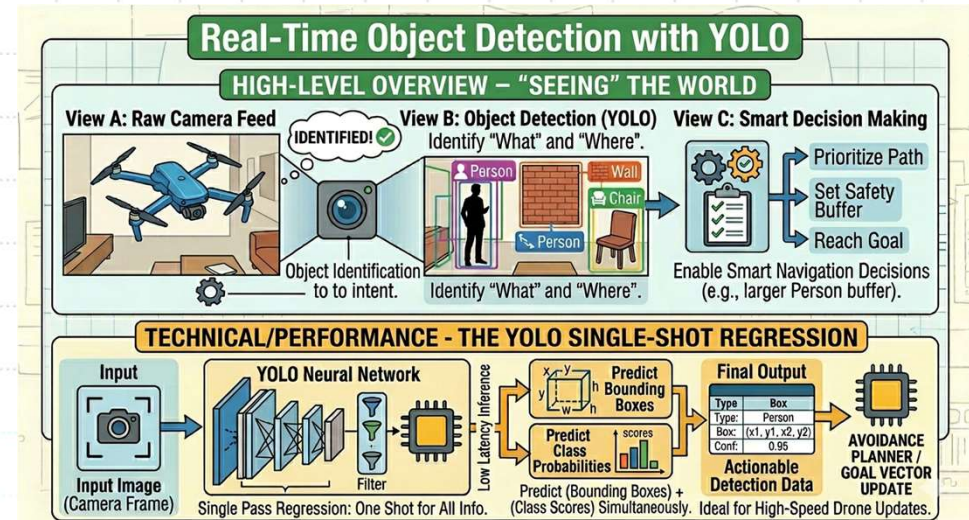
- Partition the area into equal square grid.
- Plan waypoints to make quadcopter move like a lawnmower or based some search algorithm (like breadth-first-search or depth-first-search) such that the quadcopter moves in straight line, and shift sideways at boundary.
- Very likely to get stuck.
- Keep track of visited grid.
- When stuck, use wall following strategy to track back using the visited grid until back to a grid that has unexplored directions.



Object Detection

Object Detection using Yolo

- Qualifier requires your code to detect specific object to earn points.
- Object detection allows the quadcopter to not only "see" an obstacle but to identify exactly what it is, such as a person, a wall, or another drone.
- By using the YOLO (You Only Look Once) algorithm, the system can detect multiple objects in real-time with high accuracy, enabling the drone to make split-second navigation decisions.
- We utilize YOLO for object detection because it treats detection as a single regression problem, processing the entire image in one pass to predict bounding boxes and class probabilities simultaneously.
- This single-shot approach is ideal for quadcopters, providing the low-latency inference required to update avoidance vectors at high speeds.



Getting Gazebo Drone Image

- The Gazebo streams Image from the PX4 via gz-transport topics.
- Use **gz topic -l** to find out the topic name of the image stream.
- See example (get_video.py) on how to get camera image:
 - line 36 subscribe to the image topic with a callback function that display the image.
 - Line 8 to Line 25 is the callback function.

```
1 import time
2 import numpy as np
3 import cv2
4 # Import Gazebo transport and message bindings
5 from gz.transport13 import Node
6 from gz.msgs10.image_pb2 import Image
7
8 def image_callback(msg: Image):
9     """
10     Callback function that triggers every time a new image message is received.
11     """
12     # Gazebo images are sent as raw bytes. Convert them to a NumPy array.
13     # Typical Gazebo camera encoding is 'rgb_8' (24-bit RGB)
14     frame = np.frombuffer(msg.data, dtype=np.uint8)
15
16     # Reshape the flat array into (height, width, channels)
17     # Note: msg.step is the number of bytes per row
18     frame = frame.reshape((msg.height, msg.width, 3))
19
20     # Convert RGB (Gazebo default) to BGR (OpenCV default) for correct colors
21     frame_bgr = cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)
22
23     # Display the resulting frame
24     cv2.imshow("Gazebo Live Feed", frame_bgr)
25     cv2.waitKey(1)
26
27 def main():
28     # 1. Initialize the Gazebo Transport Node
29     node = Node()
30
31     # 2. Define the topic name (use 'gz topic -l' to find yours)
32     # Common PX4 camera topic is often /camera or /world/<world_name>/model/<model_name>/link/link/sensor/camera/image
33     image_topic = "/world/roboverse/model/x500_mono_cam_0/link/camera_link/sensor/camera/image"
34
35     # 3. Subscribe to the topic
36     if node.subscribe(Image, image_topic, image_callback):
37         print(f"Subscribed to {image_topic}. Press Ctrl+C to stop.")
38     else:
39         print(f"Failed to subscribe to {image_topic}. Is Gazebo running?")
40         return
41
42     # 4. Keep the script alive to process incoming messages
43     try:
44         while True:
45             time.sleep(0.01)
46     except KeyboardInterrupt:
47         print("Shutting down...")
48     finally:
49         cv2.destroyAllWindows()
50
51 if __name__ == "__main__":
52     main()
53
```

PLEASE refer to UseDetectorExample.py

Detector.py

- Detector.py encapsulate object detection with YOLO into a class called Detector.
- The Detector calls YOLO to check if an image contains the object of interest. The object of interest is in the model file to provide when initialize the Detector.
- Your code calls Detector's **submit_image()** with the numpy object of the image. Detector will queue the image and runs the detector in background.
- When Detector has processed an image, it will display the result on screen and store the image with the annotation of the detect object in a file if there are detection beyond the confidence_threshold. The file will be stored in folder where you run your code.
- Example code on how to use Detector.py is shown on the right:
 - Line 12: Initialize Detector with the callback function and set the confidence threshold.
 - Line 42: Call detector.submit_image to detect image that is received from the subscription.
- For your drone code, you can simply subscribe to an Image topic with a callback that call Detector submit_image like this example. Then wait for result to shown on the video window or existence of photo files.
- A basic model that can detect the canister in Qualifier Challenger will be published on discord channel.

```

1 from Detector import Detector
2 import cv2
3 import time
4 import numpy as np
5 import cv2
6 # Import Gazebo transport and message bindings
7 from gz.transport13 import Node
8 from gz.msgs10.image_pb2 import Image
9
10 class VisionApp:
11     def __init__(self):
12         self.detector = Detector(
13             model_path="yolov10n.pt",
14             confidence_threshold=0.6,
15             callback=self.on_detection,
16             num_workers=2,
17             device="cpu",
18             save_dir="./detections",
19             enable_display=True, # Spawns display thread automatically
20             display_window_name="AI View"
21         )
22
23     def on_detection(self, detections, annotated_image, context):
24         print(f"Saved: {context['saved_path']} | Objects: {[d['class_name'] for d in detections]}")
25         # UI is handled automatically by the display thread now!
26
27     def image_callback(self, msg: Image):
28         """
29         Callback function that triggers every time a new image message is received.
30         """
31         # Gazebo images are sent as raw bytes. Convert them to a NumPy array.
32         # Typical Gazebo camera encoding is 'rgb_8' (24-bit RGB)
33         frame = np.frombuffer(msg.data, dtype=np.uint8)
34
35         # Reshape the flat array into (height, width, channels)
36         # Note: msg.step is the number of bytes per row
37         frame = frame.reshape((msg.height, msg.width, 3))
38
39         # Convert RGB (Gazebo default) to BGR (OpenCV default) for correct colors
40         frame_bgr = cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)
41
42         self.detector.submit_image(frame_bgr, context={"timestamp": time.time()})
43
44     def run(self):
45         # 1. Initialize the Gazebo Transport Node
46         node = Node()
47
48         # 2. Define the topic name (use 'gz topic -l' to find yours)
49         # Common PX4 camera topic is often /camera or /world/<world_name>/model/<model_name>/link/link/sensor/camera/image
50         image_topic = "/world/roboverse/model/x500_vision_0/link/camera_link/sensor/IMX214/image"
51
52         # 3. Subscribe to the topic
53         if node.subscribe(Image, image_topic, self.image_callback):
54             print(f"Subscribed to {image_topic}. Press Ctrl+C to stop.")
55         else:
56             print(f"Failed to subscribe to {image_topic}. Is Gazebo running?")
57         return
58
59     try:
60         while True:
61             time.sleep(0.01)
62         finally:
63             self.detector.stop()
64
65 if __name__ == "__main__":
66     VisionApp().run()

```

PLEASE refer to Train_YOLO_Models.ipynb

Use Train_YOLO_Models.ipynb to train custom model

- If you are interested to train your own and better model to detect the cannister in Qualifier challenge, use the Train_YOLO_Models.ipynb to learn.
- Google Colab provides a virtual machine in your browser complete with a Linux OS, filesystem, Python environment, and best of all, a free GPU via a colab jupyter notebook.
- We use a colab notebook to install PyTorch and Ultralytics in a colab environment and use it to train our model where simply click the Play button on sections of code in this notebook to execute them on the virtual machine.
- How?
 - Access Google Colab using a gmail account.
 - Open Train_YOLO_Models.ipynb notebook using Google Colab.
 - Follow the instructions in the notebook and click Play button accordingly to train a model using YOLO.

https://colab.research.google.com/drive/1Kiehs06l2kxCM8Z8aQs5e3JlfpTcTMS

Train_YOLO_Models.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Train YOLO Models in Google Colab for use with hula drone

Introduction

This notebook uses [Ultralytics](#) to train YOLOv8 object detection models with a custom dataset. At the end of this Colab, you'll have a custom YOLO model that you can run on windows PC using onnxdetector.

Important note: This notebook will be continuously updated to make sure it works with newer versions of Ultralytics and YOLO. Always follow the notebook!

Working in Colab

Google Colab provides a virtual machine in your browser complete with a Linux OS, filesystem, Python environment, and best of all, a free GPU. We'll install PyTorch and Ultralytics in this environment and use it to train our model. Simply click the Play button on sections of code in this notebook to execute them on the virtual machine.

Navigation

To navigate this notebook, use the table of contents in the left sidebar to jump from section to section.

Verify NVIDIA GPU Availability

Make sure you're using a GPU-equipped machine by going to "Runtime" -> "Change runtime type" in the top menu bar, and then selecting one of the GPU options in the Hardware accelerator section. Click Play on the following code block to verify that the NVIDIA GPU is present and ready for training.

```
[ ] Invidia-smi
```

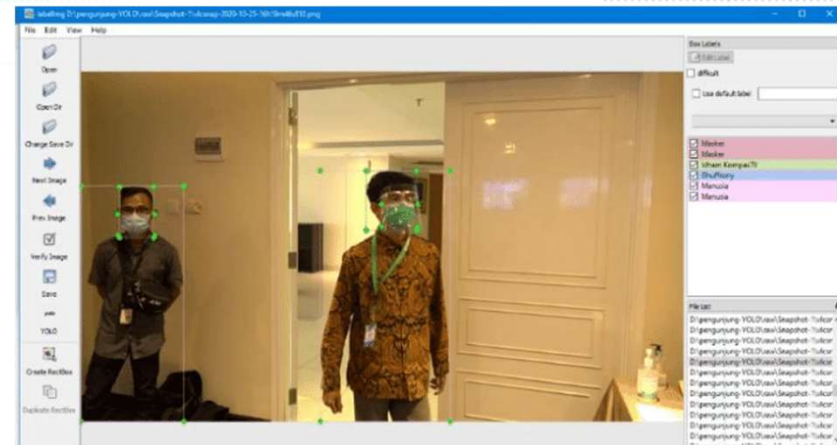
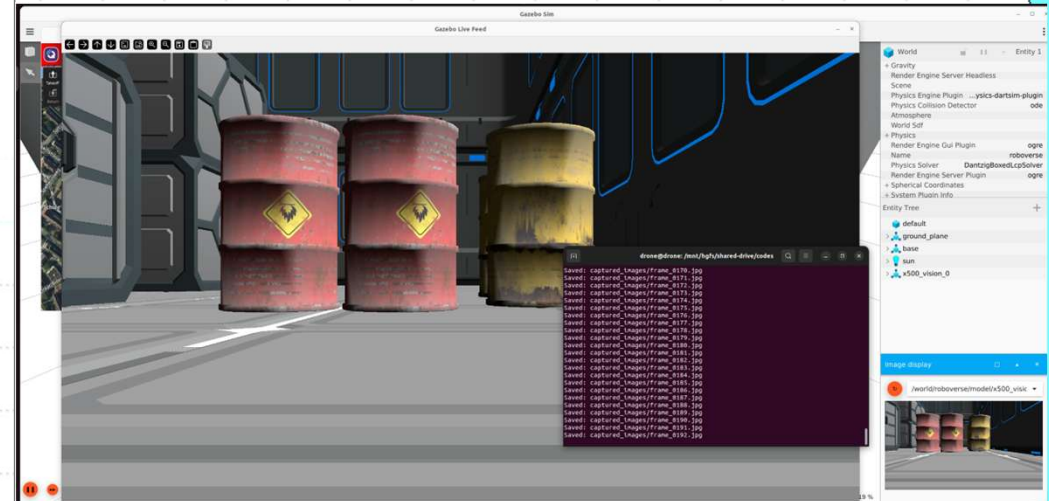
1. Gather and Label Training Images

Before we start training, we need to gather and label images that will be used for training the object detection model. A good starting point for a proof-of-concept model is 200 images. The training images should have random objects in the image along with the desired objects, and should have a variety of backgrounds and lighting conditions. See video to learn more : <https://youtu.be/v0ssiQY6cfg2>

PLEASE refer to save_photo.py

Gather and Label Training Images

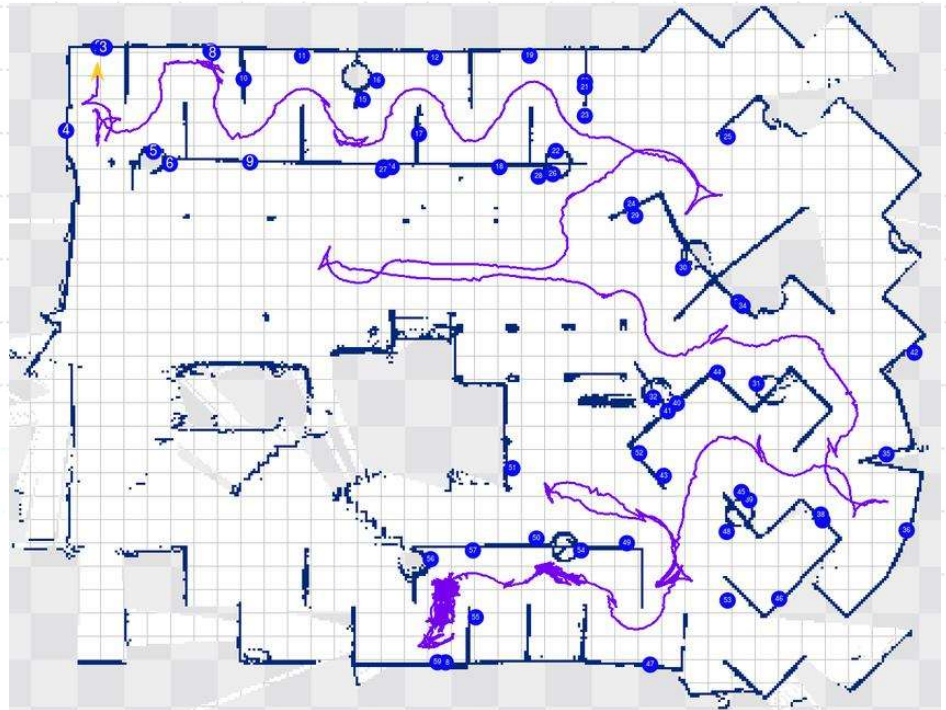
- Before custom training, use the drone's camera to take images of the desired objects that will be used for training your custom object detection model.
 - Use save_photo.py which subscribe to image topic and save all images (frame by frame) to files whenever it receives.
- The training images should have random objects in the image along with the desired objects, and should have a variety of backgrounds and lighting conditions. If possible, include actual deployment conditions.
- Then label the desired objects within the images to tell the model how does the desired objects look like.
- An image can have multiple labels.
- There are several tools for labeling images such as labeling which is free and simple.



Occupancy Grid

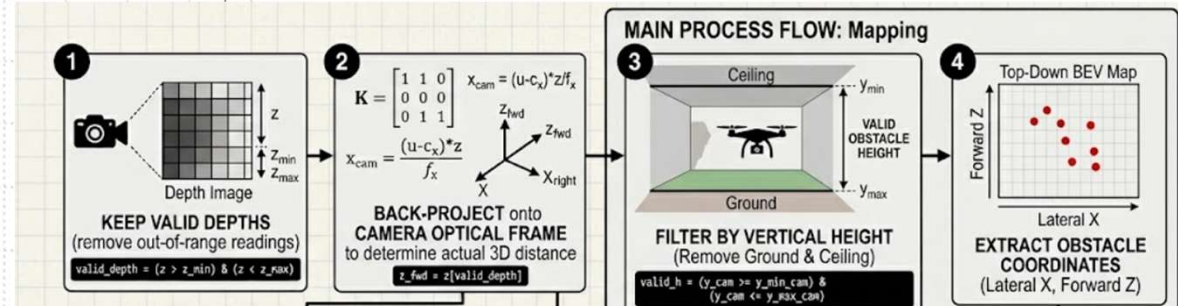
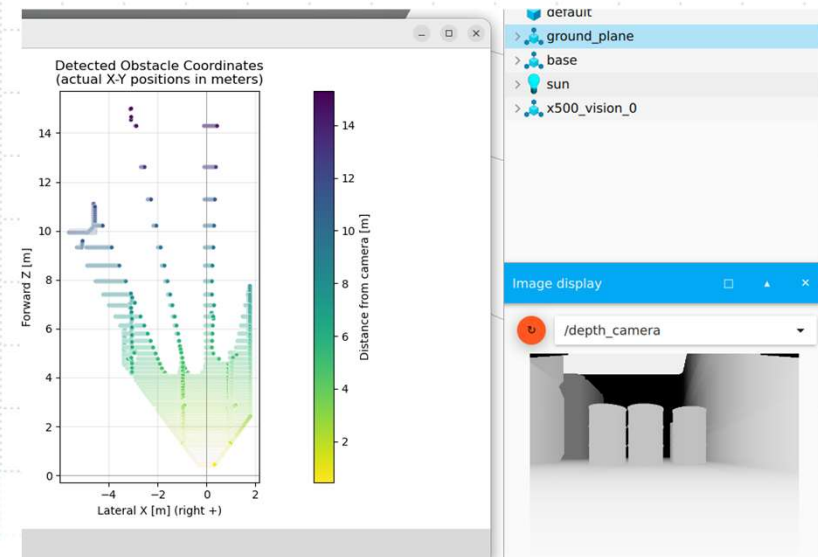
Occupancy Grid

- As the quadcopter explores the area, it collected data about accessible and un-accessible grids.
- These grid can be used to create map of the entire area in the form of occupancy grid which is useful to calculating path towards a destination.
- Qualifier Challenge does not require team to generate occupancy grid maps.
- Learning an occupancy grid is the right next step towards the Final Challenge.
- But the trick is doing it consistently with your camera frame, NED frame, and noisy depth. Most failures I see aren't from the math—they're from frame mismatch and bad update logic.
 - The solutions to these challenges will be discussed in the Supplementary Materials.



Using depth image to generate a top-down local map of a grid

- Leverage depth image to remember the position of obstacles, using a top-down local occupancy map.
- A local map that describes from the perspective of drone's current position.
- The function **depth_to_xy_map** in **top_down.py** process the depth image and output top down map using the camera intrinsic to determine the distance of obstacle.
- As the drone avoid the obstacles, repeatedly call this function and combine the map output into a global map.
- Global map is only possible when the code can know the position and orientation of the drone as it flies.



Extending the depth_to_xy_map function into a global mapper

- Globalmapper.py uses depth_to_xy_map to combine local maps into a global map.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.colors import ListedColormap, BoundaryNorm
4 from top_down import depth_to_xy_map
5 from depth_receiver import DepthReceiver
6 import time
7 from drone_control import Drone
8 import asyncio
9 from get_position_with_task import SharedState, position_monitor_task, run
10
11
12 class GlobalMapper:
13     """
14     Incremental top-down occupancy grid mapper using NED (North-East-Down) pose.
15
16     Coordinate Conventions:
17     - Pose: {'north': float, 'east': float, 'yaw': float}
18     - Yaw: radians, clockwise from North (standard NED heading)
19     - Camera: Forward-facing, level mount assumed (X_cam=right, Z_cam=forward)
20     - Grid: row=North, col=East (matches standard map orientation)
21     """
22     def __init__(self, K,
23                 cam_height=1.0, obs_h_min=0.1, obs_h_max=1.5,
24                 z_min=0.2, z_max=15.0,
25                 yaw_in_degrees=False, yaw_clockwise=True, yaw_smoothing=0.7):
26         self.K = K
27         self.cam_height = cam_height
28         self.obs_h_min = obs_h_min
29         self.obs_h_max = obs_h_max
30         self.z_min = z_min
31         self.z_max = z_max
32
33         # Yaw handling
34         self.yaw_in_degrees = yaw_in_degrees
35         self.yaw_clockwise = yaw_clockwise
36         self.yaw_smoothing = yaw_smoothing
37         self.last_yaw_rad = 0.0
38         self.first_frame = True
39
40         # Global point storage: (N, 2) array [north, east] in meters
41         self.global_points = np.empty((0, 2), dtype=np.float32)
42
43     def _local_to_ned_global(self, local_xy, north, east, yaw_rad):
44         """Transform local (X_cam=right, Z_cam=forward) to NED global (north, east)"""
45         X_cam = local_xy[:, 0]
46         Z_cam = local_xy[:, 1]
47         c, s = np.cos(yaw_rad), np.sin(yaw_rad)
48
49         north_global = north + Z_cam * c - X_cam * s
50         east_global = east + Z_cam * s + X_cam * c
51         return np.column_stack([north_global, east_global])
52
53     def update_frame(self, depth_img, pose):
54         north = pose['north']
55         east = pose['east']
56         raw_yaw = pose['yaw']
57
58         # 1. Yaw conversion & smoothing
59         if self.yaw_in_degrees:
60             raw_yaw = np.deg2rad(raw_yaw)
61         if not self.yaw_clockwise:
```

THANK YOU